

Programming Language Principles, Design, and Implementation

Lecture 4 - Untyped lambda-calculus

Sean Moss

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ **Functional languages**
 - ▶ Operational semantics
 - ▶ Type systems
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

The untyped λ -calculus

- ▶ Syntax
 - ▶ Variables (free & bound)
 - ▶ α -equivalence
 - ▶ β -equality
 - ▶ β -reduction
 - ▶ Normal forms
 - ▶ Booleans and Church numerals
 - ▶ Extensions to the calculus
-
- ▶ Formative assignment out tomorrow!

Recap: syntax

The (pure, untyped) λ -calculus has 3 constructs only:

- ▶ **variables**: $x, y, z, f, g, a, b, \dots, x_1, x_2, \dots \in \mathcal{V}$
- ▶ **functions** called “lambda abstractions”
- ▶ **application**

Fix an infinite set $\mathcal{V}$ of variables.

The following Backus-Naur form inductively defines the syntax of the λ -calculus:

$$M ::= x \mid \lambda x.M \mid M M$$

We also use parentheses to avoid ambiguities and for readability.

α -equivalence

Bound variable names are not important and can be renamed without changing the meaning of a term.

Two terms M and N are **α -equivalent** – denoted $M \equiv_{\alpha} N$ – if they only differ by their bound variables, i.e.:

- ▶ M and N have the same tree structure up to leaves
- ▶ M and N have the same occurrences of free variables
- ▶ one superscript assignment can be used to label both M and N

Examples:

- ▶ $\lambda x.x \equiv_{\alpha} \lambda y.y$
- ▶ $\lambda x.xx \not\equiv_{\alpha} \lambda x.x$ – not the same tree structure
- ▶ $x(\lambda x.yx) \equiv_{\alpha} x(\lambda z.yz)$
- ▶ $x(\lambda x.yx) \not\equiv_{\alpha} x(\lambda y.yy)$ – not the same occurrences of free variables

In practice we usually only consider λ -terms up to α -equivalence.

Capture-avoiding substitution

We want a **substitution operation** – denoted $M[x \backslash N]$ – that substitutes the term N for the free occurrences of the variable x in M .

This should be like running M in an environment where x is a name for the program N .

- ! Straightforward textual substitution is not appropriate since free variables in N could become ‘captured’.
- ! Substitution should respect α -equivalence.

E.g. in $(\lambda x. y \ x)[y \backslash xz]$ the x in $x \ z$ should continue to refer to an x to be supplied by the environment, it should not become bound by the λx .

Therefore we define **capture-avoiding substitution**.

WARNING: capture-avoiding substitution is only defined up to α -equivalence.

Capture-avoiding substitution

WARNING: for convenience, we require that, before computing $M[x \setminus N]$, the bound variables of M be disjoint from $\text{FV}(N) \cup \{x\}$.

This might require replacing M by something α -equivalent.

Substitution is defined recursively as follows:

$$(1) \quad x[x \setminus N] = N$$

$$(2) \quad y[x \setminus N] = y \\ \text{if } x \neq y$$

$$(3) \quad (\lambda y. M)[x \setminus N] = \lambda y. (M[x \setminus N])$$

$$(4) \quad (M_1 M_2)[x \setminus N] = M_1[x \setminus N] M_2[x \setminus N]$$

By the above WARNING, in case (3) we know $y \notin \text{FV}(N)$ and $y \neq x$.

- ▶ The former means we avoid ‘capturing’ the free variables of N
- ▶ The latter means we avoid substituting N for any bound variables.

Capture-avoiding substitution

Examples:

- ▶ $(xx)[x \setminus (\lambda y.y)] = (\lambda y.y)(\lambda y.y)$
- ▶ $(\lambda y.x)[x \setminus (\lambda y.y)] = \lambda y.\lambda y.y$
- ▶ $(\lambda y.yx)[x \setminus xx] = \lambda y.y(xx)$
- ▶ $(\lambda y.yx)[x \setminus yy] = \lambda z.z(yy)$
 - ▶ Note that we have silently converted $\lambda y.yx$ into $\lambda z.zx$ before substituting because $y \in \text{FV}(yy)$
 - ▶ Failing to α -convert would return the wrong result $\lambda y.y(yy)$ where the free ys from yy have been “captured” by the λ , i.e., the free ys are now bound.

β -equality

We now use substitution to define the λ -calculus's main “computation” rule called the β -conversion rule.

We say that M and N are β -equal – denoted $M =_\beta N$ – if the pair can be derived from the following rules:

β -conversion	$(\lambda x.M) N =_\beta M[x \backslash N]$
reflexivity	$M =_\beta M$
symmetry	if $M =_\beta N$ then $N =_\beta M$
transitivity	if $(M =_\beta N \text{ and } N =_\beta P)$ then $M =_\beta P$
left-apply	if $M =_\beta N$ then $M P =_\beta N P$
right-apply	if $M =_\beta N$ then $P M =_\beta P N$
lambda	if $M =_\beta N$ then $\lambda x.M =_\beta \lambda x.N$

Example: $(\lambda f.\lambda x.f x) (\lambda y.y) =_\beta \lambda x.(\lambda y.y)x =_\beta \lambda x.x$

“The λ -calculus” usually means the theory of β -equality between α -equivalence classes of λ -terms.

β -reduction

Usually we think of computation as being directed. We might instead want to talk about β -reduction.

$$(\lambda f. \lambda x. f x) (\lambda y. y) \mapsto_{\beta} \lambda x. (\lambda y. y) x \mapsto_{\beta} \lambda x. x$$

One-step β -reduction is the relation $\mapsto_{\beta}$ axiomatized by the rules:

β -conversion	$(\lambda x. M) N \mapsto_{\beta} M[x \backslash N]$
left-apply	if $M \mapsto_{\beta} N$ then $M P \mapsto_{\beta} N P$
right-apply	if $M \mapsto_{\beta} N$ then $P M \mapsto_{\beta} P N$
lambda	if $M \mapsto_{\beta} N$ then $\lambda x. M \mapsto_{\beta} \lambda x. N$

A (sub)term of the form $(\lambda x. M)N$ is called a β -redex (“reducible expression”). The relation $M \mapsto_{\beta} N$ means that N is obtained from M by reducing one β -redex **somewhere** in M

Examples

- ▶ $(\lambda x. x x)(\lambda x. x x) \mapsto_{\beta} (\lambda x. x x)(\lambda x. x x)$
- ▶ $(\lambda f. \lambda x. f x)(x x) \mapsto_{\beta} \lambda y. (x x) y$

Note: the first term reduces to itself

β -reduction

Depending on which redex we choose to reduce, we obtain 2 different terms:

- ▶ $(\lambda x. (\lambda y. y) x x) (\lambda z. z) \mapsto_{\beta} (\lambda y. y) (\lambda z. z) (\lambda z. z) \mapsto_{\beta} (\lambda z. z) (\lambda z. z)$
- ▶ $(\lambda x. (\lambda y. y) x x) (\lambda z. z) \mapsto_{\beta} (\lambda x. x x) (\lambda z. z) \mapsto_{\beta} (\lambda z. z) (\lambda z. z)$

... but both terms β -reduce to the same term

We have not specified any evaluation order yet (covered next week)

However, the order in which we reduce β -redexes does not matter here in a very precise sense:

Let $\mapsto_{\beta}^*$ be the reflexive and transitive closure of $\mapsto_{\beta}$

Theorem (confluence or Church-Rosser): If $M \mapsto_{\beta}^* N$ and $M \mapsto_{\beta}^* P$ then there exists a Q such that $N \mapsto_{\beta}^* Q$ and $P \mapsto_{\beta}^* Q$

Corollary: $M =_{\beta} N$ iff there exists P with $M \mapsto_{\beta}^* P$ and $N \mapsto_{\beta}^* P$.

Normal forms

A λ -term M with no β -redexes is said to be in **β -normal form**.

- ▶ Equivalently, there does not exist N with $M \mapsto_{\beta} N$.
- ▶ Equivalently, for all N , $M (\mapsto_{\beta})^k N$ iff $k = 0$ and $M = N$.

Examples:

$x \quad x y \quad \lambda x.x \quad \lambda x.\lambda y.x \quad \lambda x.\lambda y.y \quad x (\lambda y.y)$

Corollary of the Church-Rosser Theorem: For every M there is at most one β -normal form N with $M \mapsto_{\beta}^* N$ (or indeed with $M =_{\beta} N$).

In particular, α -inequivalent β -normal forms are never β -equal.

Possible interpretation: β -normal forms are ‘values’. The ‘value’ of a λ -term is its unique β -normal form (if it exists). ‘Evaluating’ a λ -term means trying to convert it to normal form.

But this is **not** quite analogous to what Haskell does.

Example: encoding Booleans in λ -calculus

$T = \lambda x. \lambda y. x$
 $F = \lambda x. \lambda y. y$
 $\text{and} = \lambda a. \lambda b. a \ b \ F$
 $\text{or} = \lambda a. \lambda b. a \ T \ b$
 $\text{not} = \lambda a. a \ F \ T$

$\text{or } T \ F$
 $\equiv (\lambda a. \lambda b. a \ T \ b) \ T \ F$
 $(\mapsto_{\beta})^2 \ T \ T \ F$
 $\equiv (\lambda x. \lambda y. x) \ T \ F$
 $\mapsto_{\beta} (\lambda y. T) \ F$
 $\mapsto_{\beta} T$

$\text{or } F \ F$
 $\equiv (\lambda a. \lambda b. a \ T \ b) \ F \ F$
 $(\mapsto_{\beta})^2 \ F \ T \ F$
 $\equiv (\lambda x. \lambda y. y) \ T \ F$
 $\mapsto_{\beta} (\lambda y. y) \ F$
 $\mapsto_{\beta} F$

Example: Church numerals

$$\begin{aligned}\underline{0} &= \lambda f. \lambda x. x \\ \underline{1} &= \lambda f. \lambda x. f \ x \\ \underline{2} &= \lambda f. \lambda x. f(fx) \\ &\dots \\ \text{succ} &= \lambda a. \lambda f. \lambda x. f \ (a \ f \ x) \\ \text{add} &= \lambda a. \lambda b. \lambda f. \lambda x. a \ f \ (b \ f \ x)\end{aligned}$$

Exercise: Show that $\underline{(n + 1)} \ f \ x =_{\beta} f \ (\underline{n} \ f \ x)$.

Let's see how **succ** behaves as a function applied to numerals.

$$\begin{aligned}\text{succ } \underline{n} &\mapsto_{\beta} \lambda f. \lambda x. f \ (\underline{n} \ f \ x) \\ &=_{\beta} \lambda f. \lambda x. \underline{(n + 1)} \ f \ x \\ &\mapsto_{\beta}^* \underline{(n + 1)}\end{aligned}$$

Therefore $\text{succ } \underline{n} \mapsto_{\beta}^* \underline{(n + 1)}$.

Fixed point combinators

If M is a λ -term, a **fixed point** of M is a λ -term N with

$$M N =_{\beta} N.$$

A **fixed point combinator** is a closed λ -term **fix** such that, for all M , **fix** M is a fixed point of M , i.e.

$$M (\text{fix } M) =_{\beta} \text{fix } M.$$

Curry's Y combinator is defined as

$$Y = \lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))$$

and is indeed a fixed point combinator.

Also, **Turing's Θ combinator**, defined by

$$\Theta = (\lambda xy. y(xxy))(\lambda xy. y(xxy)),$$

actually satisfies

$$\Theta M \mapsto_{\beta}^* M (\Theta M).$$

Exercise: Show that $\Theta (\lambda y. xy)$ is a 'long list' of x 's.

Designing extensions to λ -calculus I

Constants vs variables

- ▶ It's strange that we only work with **encodings** of Booleans and natural numbers.
- ▶ More natural to have primitive terms for truth values, etc.
- ▶ Could fix variables like $b_{\text{True}}, b_{\text{False}}, b_{\text{and}}, \dots$ and $n_0, n_{\text{succ}}, \dots$, but:
- ▶ we would never λ -abstract them.
- ▶ we might want **extra reduction rules**.

Example: λ -calculus with Booleans

Extend grammar with three primitive constants: 'true', 'false', 'if-then-else'.

$$M ::= x \mid \lambda x.M \mid M M \mid \text{T} \mid \text{F} \mid \text{ite}$$

Extend reduction rules:

$$\begin{aligned}(\lambda x.M) N &\mapsto M[x \backslash N] \\ \text{iteT } M N &\mapsto M \\ \text{iteF } M N &\mapsto N\end{aligned}$$

Designing extensions to λ -calculus II

Only constants vs more general term constructs

- ▶ We suggested $M ::= x \mid \lambda x.M \mid M M \mid T \mid F \mid \text{ite}.$
- ▶ But then 'ite $x y$ ' is like an incomplete sentence.

Example: alternative λ -calculus with Booleans

Extend grammar with

- ▶ two primitive constants ('nullary term formers'): 'true', 'false'
- ▶ one ternary **term constructor**: 'if-then-else'.

$$M ::= x \mid \lambda x.M \mid M M \mid T \mid F \mid \text{ifthenelse}(M, M, M)$$

Extend reduction rules:

$$(\lambda x.M) N \mapsto M[x \backslash N]$$

$$\text{ifthenelse}(T, M, N) \mapsto M$$

$$\text{ifthenelse}(F, M, N) \mapsto N$$

WARNING: The expression $\text{ifthenelse}(M, N, P)$ is not a sequence of applications. In particular, it does not allow 'partial applications'.

Conclusion

What did we cover today?

- ▶ Capture-avoiding substitution
- ▶ β -equality, β -reduction, β -normal forms
- ▶ Booleans and Church numerals
- ▶ Fixed point combinators
- ▶ Extensions to the calculus

Further reading:

- ▶ Chapter 5 of “Benjamin C. Pierce. Types and Programming Languages. MIT Press, 2002.”

Next time?

- ▶ Semantics
- ▶ Example: untyped arithmetic expressions